kata – Describe katas with pictograms

Lukas Heindl
oss.heindl+latex@protonmail.com
Github: https://github.com/atticus-sullivan/kata/

0.1.0 from 2026/03/23

Abstract

The `kata` package provides a compact and structured way to typeset martial arts katas[1] (forms) in LaTeX. It models a kata as a sequence of steps, each combining the stance hand/foot technique, and transition direction, and supports customizing the resulting pictograms for clear visual representation.

1 Introduction

In many martial arts such as Karate, a kata is a predefined sequence of movements [2].

The `kata` package aims to provide a simple, structured way to describe katas in LaTeX. Each step of a kata is expressed as a step consisting of

1. the stance with direction
2. a technique with direction
3. the direction of movement or rotation into the next step

The package also offers customizing the resulting pictograms, allowing visual annotation of sequences, such as highlighting repetitions or emphasizing specific actions.

1.1 Simple Example

Note: The output looks slightly different in the example's output here. Usually, the section is a proper section and the marginline is really show in the margin.

Mocks are only used here in the docs avoid problems showing this inside the example box <*doc>

A simple example without customizations (but includes flip)

```
\begin{kata}[width=3]{Simple Example}{blue}
  Some explanation -- normal \LaTeX code
  \parindent=0pt
```

[1] https://en.wikipedia.org/wiki/Kata
[2] https://en.wikipedia.org/wiki/Kata

```
  \begin{step}
    \stance{stance A}{0}
    \waza{technique A}{-90}
    \dir{center}{90}
  \end{step}
  \begin{step}
    \stance{stance A}{0}
    \dir{90}{-90}
  \end{step}
  \begin{step}
    \stance{stance A}{0}[flip]
    \dir[flip]{90}{-90}
  \end{step}
\end{kata}
```

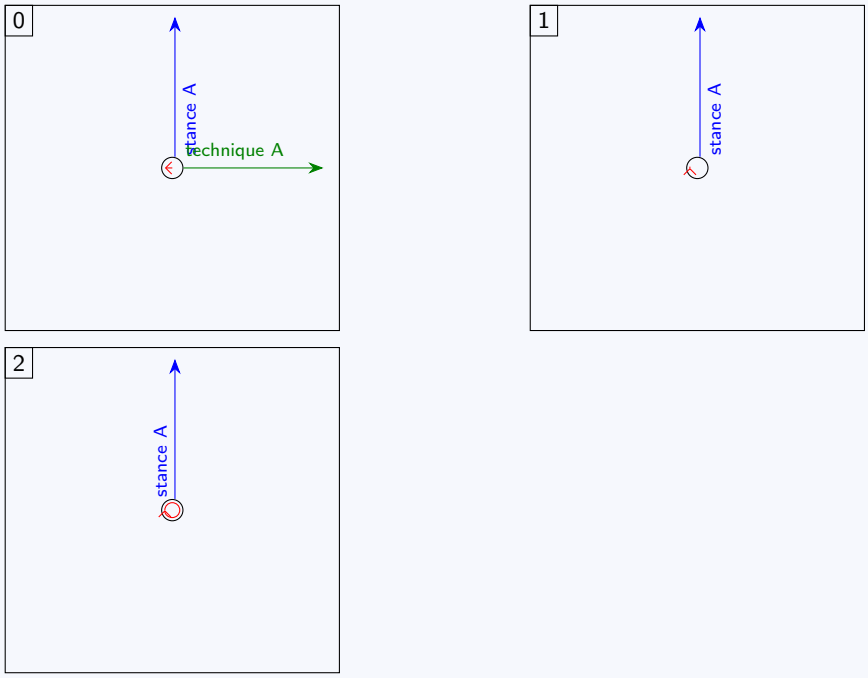
X. Simple Example

Margin would be:
(*begin marginline*)

↓ blue

(*end marginline*)

Some explanation – normal LaTeXcode



</doc>

1.2 Extended Example

Note: The output looks slightly different in the example's output here. Usually, the section is a proper section and the marginline is really show in the margin.

Mocks are only used here in the docs avoid problems showing this inside the

example box <*doc>

Extended example with all kinds of possible customizations

```
\installMocks % only used here to avoid problems showing inside tcolorbox
\begin{kata}[width=3,color={waza=orange}]{Extended Example}{blue}
  Some explanation -- normal \LaTeX code
  \parindent=0pt

  \begin{step}[color={stance=pink}]
    \stance{stance A}{0}
    \waza{technique A}{-90}[color=black]
    \dir{center}{90}
  \end{step}
  \kataSetup{color={dir=brown}}
  {
    \kataSetup{border preset={arrow up,accent=green}}
    \begin{step}[]
      \stance{stance B}{180}
      \waza{technique B}{90}
      \dir{90}{180}
    \end{step}
    \renewcommand\kataStepCnt[1]{\roman{#1}}
    \begin{step}
      \stance{stance B}{180}
      \waza[R]{technique B}{90}[flip]
      \dir[flip]{90}{180}
    \end{step}
  }
  \pgfkeys{
    /kata/style/step counter/.append style={xshift=1em},
    /kata/style/border/.append style={/kata/preset/helpers/arrowUD=45},
  }
  \begin{step}
    \stance{stance C}{180}
    \waza{technique D}{0}
  \end{step}
\end{kata}
```

only used here to avoid problems showing inside tcolorbox

X. Extended Example

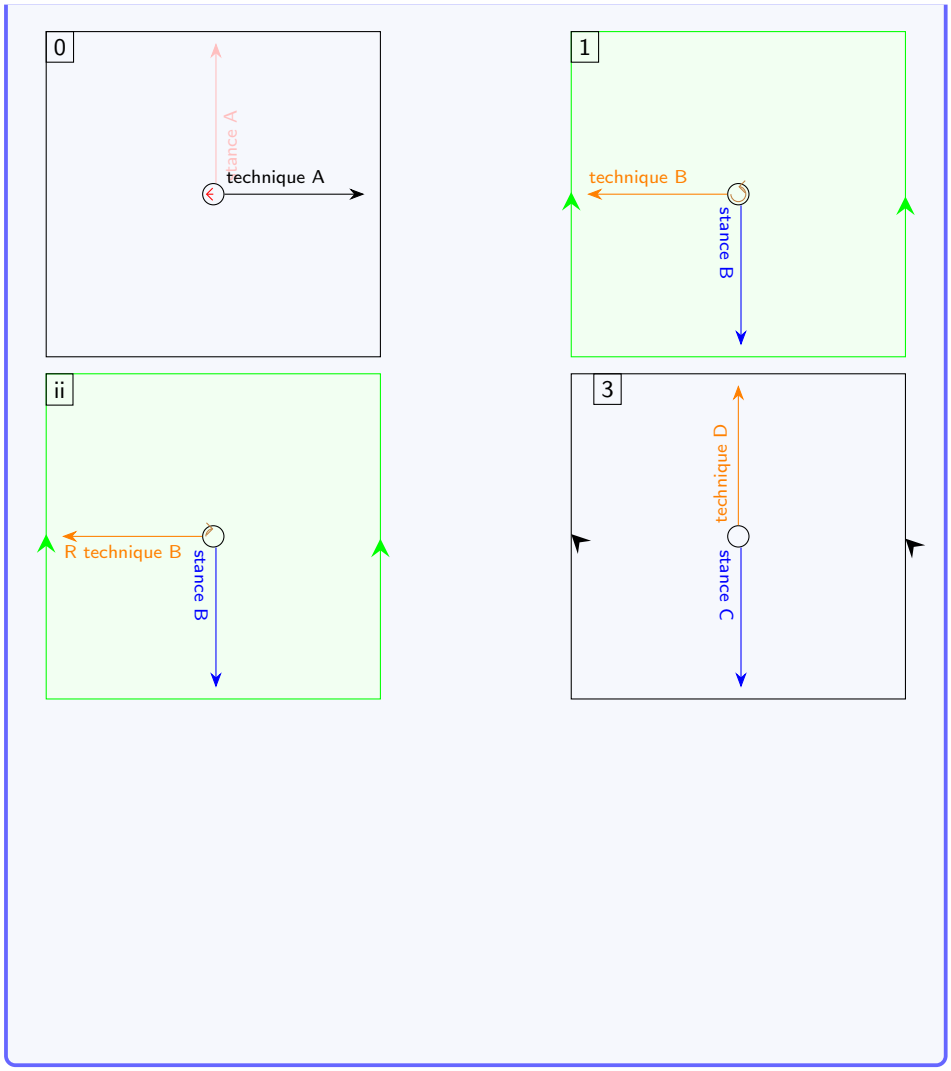
Margin would be:

(*begin marginline*)

↓ blue

(*end marginline*)

Some explanation – normal LaTeXcode



</doc>

2 Implementation

```
\RequirePackage{memoize}
\RequirePackage{pgfkeys}
\RequirePackage{pgfopts}
\RequirePackage{coloredbelts}
\RequirePackage{tikz}
\usetikzlibrary{calc,positioning,fit,arrows.meta}
\usetikzlibrary{decorations.markings,decorations.pathmorphing,backgrounds}
```

Define marginline in case it isn't already defined (so the package works without KOMA-classes as well).

```
\AddToHook {begindocument} [kata/provideMarginline] {%
  \providecommand\marginline[1]{%
    \marginpar[{\raggedleft #1}]{\raggedright #1}%
```

```
  }%
}
```

Define own string compare for equality so the package works without KOMA as well.

```
\ExpandArgs {nnc} \UseName {cs_set_eq:NN} {\kata@ifstr} {str_if_eq:eeTF}
```

2.1 General pgfkeys setup

```
\pgfkeys{
```

for package wide settings

```
  /kata/setup/.is family,
```

for settings regarding for the kata env

```
  /kata/kata/.is family,
```

for settings regarding for the step env

```
  /kata/step/.is family,
```

for settings regarding for the stance command

```
  /kata/stance/.is family,
```

for settings regarding for the waza command

```
  /kata/waza/.is family,
```

for settings regarding for the dir command

```
  /kata/dir/.is family,
```

for settings regarding coloring

```
  /kata/color/.is family,
```

internal style definitions

```
  /kata/style/.is family,
```

presets for better usability

```
  /kata/preset/border/.is family,
```

forwards in order to DRY

```
  /kata/forward/.is family,
}
```

2.2 Set up pgfkeys-forwards

```
\pgfkeys{
  /kata/forward,
```

Common keys which simply delegate to other keys

```
  delegate common/.style={
    stance/.style = {/kata/stance,##1},
    waza/.style = {/kata/waza,##1},
    dir/.style = {/kata/dir,##1},
    color/.style = {/kata/color,##1},
  },
```

Common keys which append to styles

```
  append common/.style={
    head style/.style = {/kata/style/head/.append style = {##1}},
    border style/.style = {/kata/style/border/.append style = {##1}},
    border preset/.forward to = /kata/style/border preset,
    step counter style/.style = {/kata/style/step counter/.append style = {##1}},
    step phantom style/.style = {/kata/style/step phantom/.append style = {##1}},
  },
}
```

```
%
%
% \subsection{Package wide options}
% Defer keys in setup to the respective locations
%    \begin{macrocode}
\pgfkeys{
  /kata/setup,
  width/.style = {/kata/kata,width={#1}},
  definealiases/.code = {%
    \AddToHook {begindocument} [kata/aliases] {%
      \kataAliases%
    }%
  },
  /kata/forward/delegate common,
  /kata/forward/append common,
}
```

Setup package option parsing

```
\ProcessPgfPackageOptions{/kata/setup}
```

A setup command for setting package options later

\kataSetup

```
\NewDocumentCommand{\kataSetup}{m}{%
  \pgfkeys{/kata/setup,#1}%
  \ignorespaces%
}%
```

2.3 Optionally define command aliases

\kataAliases

```
\newif\ifkata@aliasesdefined
\kata@aliasesdefinedfalse
\NewDocumentCommand{\kataAliases}{}{
  \ifkata@aliasesdefined\else%
    \kata@aliasesdefinedtrue%
    \newcommand{\kata@warningdefined}[1]{\PackageWarning{kata}{Macro ##1 is already defined, di
    \@ifundefined{stance}{\let\stance\kataStance}{\kata@warningdefined{\string\stance}}%
    \@ifundefined{waza}{\let\waza\kataWaza}{\kata@warningdefined{\string\waza}}%
    \@ifundefined{dir}{\let\dir\kataDir}{\kata@warningdefined{\string\dir}}%
    \@ifundefined{kiai}{\let\kiai\kataKiai}{\kata@warningdefined{\string\kiai}}%
    % environment begin/end macros
    \@ifundefined{step}{%
      \let\step\kataStep%
      \let\endstep\endkataStep%
    }{%
      \PackageWarning{kata}{Environment/Macro \string\step is already defined, did not define t
    }%
  \fi%
}
```

2.4 Core internals

A counter for each pictogram to enable referencing.

```
\newcounter{kata@step}[section]
```

`\kata@requirestep` enforces something is run within the step environment (helper)

```
\newif\ifkata@instep
\NewDocumentCommand{\kata@requirestep}{m}{%
  \ifkata@instep\else
    \PackageError{kata}{#1 used outside step environment}{}
  \fi
}
```

`\kata@requirekata` enforces something is run within the kata environment (helper)

```
\newif\ifkata@inkata
\NewDocumentCommand{\kata@requirekata}{m}{%
  \ifkata@inkata\else
    \PackageError{kata}{#1 used outside kata environment}{}
  \fi
}
```

TODO: is this needed? Or should the keys just set a style instead of setting a conditional?

```
\newif\ifkata@stance@flip
\newif\ifkata@waza@flip
\newif\ifkata@dir@flip
\newif\ifkata@step@first
\newif\ifkata@step@phantom
\newcount\kata@tmpcnta
```

2.5 The kata environment

```
\pgfkeys{
  /kata/kata,
  width/.store in = \kata@width,
  width = 6,
  width/.value required,
  /kata/forward/delegate common,
  /kata/forward/append common,
}
```

`kata` (*env.*) this environment is basically a container for a sequence of steps. Arguments:

1. optional: keys from `/kata/kata`
2. kata name
3. for which belt / marginline
4. optional: belt color (for drawing), if empty -> not drawn, if not present use arg 3

```
\NewDocumentEnvironment{kata}{O{}mmo}{%
  \pgfkeys{/kata/kata,#1}%
  \section{#2}%
  \IfBlankTF{#3}{%
```

show if arg 3 is empty but arg 4 was set

```
    \IfNoValueF{#4}{\kata@beltmargin{}{#4}}%
  }{%
```

use arg 3 as fallback if arg 4 was not set

```
      \kata@beltmargin{#3}{\IfNoValueTF{#4}{#3}{#4}}%
    }%
    \setlength{\lineskip}{6pt}\noindent%
    \setcounter{kata@step}{0}%
    \kata@inkatatrue%
    \ignorespaces%
```

there is no harm in a single word/pictogram in the last line to turn off the corresponding penalty

```
    \widowpenalty=0
```

allow large larger spaces between the pictograms if this avoids drawing a picture outside of the textarea

```
    \sloppy
  }{%
```

add hidden steps to the last line. This ensures a uniform looking grid of pictograms

```
    \pgfmathtruncatemacro{\kata@missing}{mod(\kata@width - mod(\c@kata@step,\kata@width),\kata@wi
    \setcounter{kata@step}{0}%
    \ifnum\kata@missing>0
      \foreach \i in {1,...,\kata@missing} {%
        \begin{step}[phantom]\end{step} %
      }%
    \fi%
    \kata@inkatafalse%
```

avoid left-aligning the last line. Instead it should be justified in order to obtain a proper grid

```
    \parfillskip=0pt
    \par%
  }
```

`\kata@beltmargin`

```
  \NewDocumentCommand{\kata@beltmargin}{+mm}{%
    \marginline{%
      \centering\footnotesize%
      \IfBlankF{#1}{%
        $\downarrow$ #1%
      }%
```

only linebreak if both are non-empty

```
    \IfBlankF{#1}{\IfBlankF{#2}{\\}}%
    \IfBlankF{#2}{%
        \ColorBelt*[scale=0.1]{#2}%
      }%
    }%
  }
```

2.6 The step environment

```
  \pgfkeys{
    /kata/step,
    first step/.is if = kata@step@first,
    first step/.default = true,
```

```
    phantom/.is if = kata@step@phantom,
    phantom/.default = true,
    phantom = false,
    phantom/.append code={\ifkata@step@phantom\pgfkeysalso{/kata/style/step phantom}\fi},
    phantom/.style = {/kata/style/step phantom},
    /kata/forward/delegate common,
    /kata/forward/append common,
  }
```

`step` (*env.*) this environment always should wrap `kata@step`. It handles the aspects of positioning the pictogram. Arguments: 1 (optional): whether this is the first step of the kata / pgfkeys

```
\NewDocumentEnvironment{kataStep}{O{}+b}{%
  \kata@requirekata{\string\kataStep}%
  \pgfkeys{/kata/step,#1}%
  \ifkata@step@first\par\nointerlineskip\vfil\else\fi%
  \begin{kata@step}%
    #2
  \end{kata@step}%
  \ifkata@step@phantom\else\refstepcounter{kata@step}\fi%
}{}
```

`kata@step` (*env.*) the environment to draw everything else in

```
\NewDocumentEnvironment{kata@step}{}{%
  \kata@requirekata{\string\step}%
  \begin{tikzpicture}[font=\scriptsize]%
    \begin{scope}[local bounding box=output]%
      \kata@head\kata@nothing%
      \kata@insteptrue%
}{
    \kata@instepfalse%
    \end{scope}%
```

Drawing the border on the background has the advantage the rectangle can be filled and colorize the background as well.

```
    \begin{scope}[on background layer]%
      \node[/kata/style/border] (t1) {};%
      \node[/kata/style/step counter] (cnt) {\kataStepCnt{kata@step}};%
    \end{scope}%
  \end{tikzpicture}%
}
```

2.7 Specify the Content

`\kiai` draw indicator for *kiai*. Arguments:

1. (optional) distance to head TODO: not implemented yet (probably will become pgf-interface)
2. angle

```
\NewDocumentCommand{\kataKiai}{O{}m}{
  \kata@requirestep{\string\kataKiai}
  \def\base{#2+90} \def\off{30} \def\ext{40} \def\roff{4pt} \def\len{0.75}
  \draw[shorten <=10pt,CtpRed]
```

```
    ($(head)+(\base-\off:\roff)$) edge[decorate,decoration={bent,aspect=0.3,amplitude=-2.5pt}] co
    ($(head)+(\base+\off:\roff)$) edge[decorate,decoration={bent,aspect=0.3,amplitude=+2.5pt}] co
    \path (tmpA) -- (tmpB) node[midway,sloped,font={\sffamily\scriptsize},CtpRed,draw,decorate,de
  }
```

2.7.1 The stance command

```
  \pgfkeys{
    /kata/stance,
    flip/.is if = kata@stance@flip,
    flip/.default = true,
    style/.style = {/kata/style/stance/.append style = {#1}},
    label/.style = {/kata/style/stance label/.append style = {#1}},
    label flipped/.style = {/kata/style/stance label flipped/.append style = {#1}},
    color/.forward to = /kata/color/stance,
  }
```

`\kataStance` draw the arrow/path showing the stance Usage:

1. (optional) side to use (L/R) / prefix
2. name/text
3. angle
4. (optional) pgfkeys interface

```
  \NewDocumentCommand{\kataStance}{o+mmO{}}{%
    \pgfkeys{/kata/stance,#4}%
    \kata@requirestep{\string\kataStance}%
    \pgfmathsetmacro{\kata@dir}{#3+90}%
    \pgfmathsetmacro{\x}{abs(sin(\kata@dir))}%
    \pgfmathsetmacro{\y}{-abs(cos(\kata@dir))}%
    \IfNoValueTF{#1}{\def\kata@nodecontent{}}{\def\kata@nodecontent{#1 }}%
    \draw[/kata/style/stance={\x}{\y}]
      (head) -- ++(\kata@dir:2)
      node[\ifkata@stance@flip/kata/style/stance label flipped\else/kata/style/stance label\fi]
      {\hangafter=1\settowidth\hangindent{\kata@nodecontent}\kata@nodecontent#2}
    ;
  }
```

2.7.2 The waza command

```
  \pgfkeys{
    /kata/waza,
    flip/.is if = kata@waza@flip,
    flip/.default = true,
    style/.style = {/kata/style/waza/.append style = {#1}},
    label/.style = {/kata/style/waza label/.append style = {#1}},
    label flipped/.style = {/kata/style/waza label flipped/.append style = {#1}},
    color/.forward to = /kata/color/waza,
  }
```

`\kataWaza` draw the arrow/path showing a technique. Arguments:

1. (optional) side to use (L/R) / prefix

2. name/text
3. angle
4. (optional) pgfkeys interface

```
\NewDocumentCommand{\kataWaza}{o+mmO{}}{%
  \kata@requirestep{\string\kataWaza}%
  \pgfkeys{/kata/waza,#4}%
  \IfNoValueTF{#1}{\def\kata@nodecontent{}}{\def\kata@nodecontent{#1 }}%
  \pgfmathsetmacro{\kata@dir}{#3+90}%
  \draw[/kata/style/waza]
    (head) -- ++(\kata@dir:2)
    node[\ifkata@waza@flip/kata/style/waza label flipped\else/kata/style/waza label\fi]
    {\hangafter=1\settowidth\hangindent{\kata@nodecontent}\kata@nodecontent#2}
  ;
}
```

2.7.3 The dir command

```
\pgfkeys{
  /kata/dir,
  flip/.is if = kata@dir@flip,
  flip/.default = true,
  style/.style = {/kata/style/dir/.append style = {#1}},
  color/.forward to = /kata/dir/waza,
}
```

`\dir` draw a short arrow/path indicating the next turn/direction. Arguments:

1. (optional) flip / pgfkeys
2. source (Center,angle)
3. dst (angle)

```
\NewDocumentCommand{\kataDir}{O{}mm}{
  \kata@requirestep{\string\kataDir}
  \pgfkeys{/kata/dir,#1}
  \pgfmathsetmacro{\kata@out}{#3+90}
  \kata@ifstr{#2}{center}{
    \draw[/kata/style/dir] (head.center) -- ++(\kata@out:.33em);
  }{
    \pgfmathsetmacro{\kata@in}{#2+90}
    \pgfmathsetmacro{\kata@outdeltai}{\kata@out-\kata@in}
    \kata@tmpcnta=\inteval{\kata@out-\kata@in}
    \ifkata@dir@flip
      \ifnum\kata@tmpcnta<0
        \pgfmathsetmacro{\kata@outdeltaii}{360+\the\kata@tmpcnta}
      \else
        \pgfmathsetmacro{\kata@outdeltaii}{\the\kata@tmpcnta-360}
      \fi
    \else
      \pgfmathsetmacro{\kata@outdeltaii}{\the\kata@tmpcnta}
    \fi
    \draw[/kata/style/dir]
```

```
      (head.center) ++ (\kata@in:.33em)
      arc[start angle=\kata@in,delta angle=\kata@outdeltaii,radius=.33em]
    ;
  }
}
```

2.7.4 Internal elements

`\kata@nothing` ensure the box is always at least the same size (internal command + always placed)

```
\NewDocumentCommand{\kata@nothing}{}{
  \draw[draw=none] (head) -- ++(0:2.15) -- ++(180:4.3) -- ++(90:2.15) -- ++(-90:4.3);
}
```

`\kata@head` draw the circle symbolizing the head (internal command + always placed)

```
\NewDocumentCommand{\kata@head}{}{
  \node[/kata/style/head] (head) {};
}
```

2.8 Color keys

```
\pgfkeys{
  /kata/color,
  %
  stance/.store in = \kata@color@stance,
  stance/.value required,
  stance = blue,
  %
  waza/.store in = \kata@color@waza,
  waza/.value required,
  waza = green!50!black,
  %
  dir/.store in = \kata@color@dir,
  dir/.value required,
  dir = red,
  %
  head/.store in = \kata@color@head,
  head/.value required,
  head = black,
  %
  border/.store in = \kata@color@border,
  border/.value required,
  border = black,
  %
  step counter/.store in = \kata@color@stepcounter,
  step counter/.value required,
  step counter = black,
}
```

2.9 Styles

2.9.1 Styles for drawn elements

```
\pgfkeys{
  /kata/style,
  %
```

```
  head/.style = {
    circle,
    draw=\kata@color@head,
    inner xsep=0.333em,
  },
  %
  stance/.style 2 args = {
    draw=\kata@color@stance,
    -{Stealth[scale=1.2]},
    transform canvas={xshift=#1,yshift=#2},
  },
  %
  waza/.style = {
    draw=\kata@color@waza,
    -{Stealth[scale=1.2]},
  },
  %
  dir/.style = {
    draw=\kata@color@dir,
    -{Straight Barb},
  },
  %
  border/.style = {
    rectangle,
    draw=\kata@color@border,
    fit=(output),
    inner sep=0pt,
    outer sep=0pt,
    inner xsep=0.2em,
  },
  %
  border preset/.code={
    \pgfkeysalso{/kata/preset/border,#1}%
  },
  %
  step counter/.style = {
    draw=\kata@color@stepcounter,
    font=\sffamily\small,
    outer sep=0pt,
    at={(t1.north west)},
    below right,
    rectangle,
  },
}
```

2.9.2 Styles for labels

```
\pgfkeys{
  /kata/style,
  %
  stance label base/.style = {
    \kata@color@stance,
    midway,
    sloped,
    font=\sffamily\scriptsize,
```

```
    text width=1.8cm,
  },
  stance label/.style = {
    /kata/style/stance label base,
    below,
  },
  %
  stance label flipped/.style = {
    /kata/style/stance label base,
    above,
  },
  %
  waza label base/.style = {
    \kata@color@waza,
    midway,
    sloped,
    font=\sffamily\scriptsize,
    text width=1.8cm,
  },
  %
  waza label/.style = {
    /kata/style/waza label base,
    above,
  },
  %
  waza label flipped/.style = {
    /kata/style/waza label base,
    below,
  },
}
```

2.9.3 Styles for steps

```
\pgfkeys{
  /kata/style,
  %
  step phantom/.style = {
    /kata/style/border/.style={draw=none},
    /kata/style/head/.style={draw=none},
    /kata/style/step counter/.style={opacity=0},
  },
}
```

2.9.4 Presets for styles

Helpers

```
\pgfkeys{
  /kata/preset/helpers/.cd,
  arrowRL/.style={
    postaction={decorate},decoration={
      markings,
      mark=at position 0.125
        with {\pgftransformrotate{#1 - 270}\arrow[line width=2pt]{stealth}},
      mark=at position 0.625
        with {\pgftransformrotate{#1 - 270}\arrowreversed[line width=2pt]{stealth}},
    },
```

```
    },
    arrowUD/.style={
      postaction={decorate},decoration={
        markings,
        mark=at position 0.375
          with {\pgftransformrotate{#1}\arrow[line width=2pt]{stealth}},
        mark=at position 0.875
          with {\pgftransformrotate{#1}\arrowreversed[line width=2pt]{stealth}},
      },
    },
  }
```

Helpers

```
  \pgfkeys{
    /kata/preset/border,
    arrow right/.style={
      /kata/style/border/.append style={
        /kata/preset/helpers/arrowRL=-90,
      },
    },
    arrow left/.style={
      /kata/style/border/.append style={
        /kata/preset/helpers/arrowRL=+90,
      },
    },
    arrow up/.style={
      /kata/style/border/.append style={
        /kata/preset/helpers/arrowUD=0,
      },
    },
    arrow down/.style={
      /kata/style/border/.append style={
        /kata/preset/helpers/arrowUD=180,
      },
    },
    accent/.style={
      /kata/style/border/.append style={
        fill=#1!5,
        draw=#1,
      },
    },
  }
```

2.10 Misc

`\kataStepCnt` Formats the counter for the step indicator

```
\NewDocumentCommand{\kataStepCnt}{m}{\arabic{#1}}
```

2.11 Memoization

TODO: which pgfkeys to add TOOD: make optional depending on whether memoize is loaded or not

```
\mmzset{auto={kata@step}{
  memoize,
  csname meaning to context={environment kata@step\space code},
  csname meaning to context={environment kata@step end aux\space},
  csname meaning to context={nothing code},
  csname meaning to context={head code},
  csname meaning to context={stance code},
  csname meaning to context={waza code},
  csname meaning to context={dir code},
}}
```

Change History

0.1.0
General: Converted to DTX file . . 1